SLIIT

DEPARTMENT OF COMPUTER SOFTWARE ENGINEERING

FACULTY OF COMPUTING

Module - Current Trends in Software Engineering (SE4010)

| 2026 | Semester 1

Azure Microservices

Deployment Lab

Self-Guided Lab

Lab Overview

Welcome to the Azure Microservices Deployment Lab. In this hands-on lab, you will build and deploy a containerized microservices application on Microsoft Azure using industry-standard tools and best practices.

By the end of this lab you will have:

- Authenticated with Azure via the CLI
- Provisioned cloud infrastructure (Resource Group, Container Registry)
- Built and pushed Docker images to a private registry
- Deployed a containerized gateway service using Azure Container Apps
- Hosted a static frontend with Azure Static Web Apps
- Verified your deployment and cleaned up resources

Lab Architecture

The application follows a microservices pattern with three main components:

- Gateway Service – Routes incoming HTTP requests to appropriate backend services
- Static Web App – Serves the React/HTML frontend to end users
- Azure Container Registry (ACR) – Private Docker image registry

Prerequisites

Before starting this lab, ensure you have the following installed and ready:

- Azure CLI (az) – version 2.50 or later. Install from: https://aka.ms/installazurecli
- Docker Desktop – running on your local machine
- A valid Azure account with an active subscription
- Git – for cloning the sample application repository
- Node.js v18+ (required for the Static Web App frontend)

Tip: Run az --version and docker --version in your terminal to verify both tools are installed correctly before beginning.

Task 1 – Login to Azure

Objective

Authenticate your terminal session with Microsoft Azure and confirm you are operating in the correct subscription. All subsequent tasks depend on this session being active.

Background

The Azure CLI (az) uses OAuth2 device-code or browser-based authentication to obtain an access token. Once authenticated, the token is cached locally and used for subsequent commands. If you work across multiple subscriptions, it is critical to set the correct active subscription before provisioning resources.

Step-by-Step Instructions

Step 1.1 – Open a Terminal

Open a terminal window (PowerShell, Command Prompt, or Bash). Confirm the Azure CLI is available:

```
az --version
```

You should see output showing the az-cli version (2.50+) and component versions.

Step 1.2 – Authenticate with Azure

Run the following command to launch the browser-based login flow:

```
az login
```

A browser window will open. Log in using your Azure account credentials (university or personal Microsoft account). Once logged in, return to your terminal — you will see a JSON list of your subscriptions.

📝 **Note:** *If you are working in a headless or SSH environment, use az login --use-device-code instead. This gives you a code to enter at https://microsoft.com/devicelogin.*

Step 1.3 – Verify Your Account

Confirm which account and subscription are active:

```
az account show
```

Expected output fields to check:

- "user": { "name": "your.email@domain.com" } – your authenticated identity

- "name": "Your Subscription Name" – the active subscription
- "state": "Enabled" – confirms the subscription is active

Step 1.4 – (Optional) Set a Specific Subscription

If you have multiple subscriptions, set the correct one explicitly:

```
az account list --output table
az account set --subscription "<Subscription Name or ID>"
```

Verification Checklist

- ☐ az --version shows version 2.50 or higher
- ☐ az login completed without errors
- ☐ az account show shows your correct email address
- ☐ Subscription state is Enabled

⚠ **Warning:** If az login opens a browser but fails to redirect back to the terminal, copy the URL from the terminal output and open it manually in your browser.

Task 2 – Create Resource Group & Container Registry

Objective

Create an Azure Resource Group to logically contain all lab resources, and provision an Azure Container Registry (ACR) to store your Docker images privately.

Background

A Resource Group in Azure is a logical container for related resources. All resources in a group share the same lifecycle — they can be deployed, managed, and deleted together. Using a dedicated resource group for this lab makes cleanup easy at the end.

Azure Container Registry (ACR) is a private Docker registry hosted on Azure. It integrates natively with Azure Container Apps and other Azure services, so you do not need to manage credentials manually when deploying.

Step-by-Step Instructions

Step 2.1 – Create the Resource Group

Run the following command to create a new resource group named microservices-rg in the East US region:

```
az group create --name microservices-rg --location eastus
```

Expected output:

```
{
  "id": "/subscriptions/.../resourceGroups/microservices-rg",
  "location": "eastus",
  "name": "microservices-rg",
  "properties": { "provisioningState": "Succeeded" }
}
```

💡 **Tip:** Choose a region close to your users or your team. For this lab, eastus is recommended for availability and cost. Other options: westus2, westeurope, southeastasia.

Step 2.2 – Create the Container Registry

Create an Azure Container Registry with the Basic SKU (suitable for development and labs):

```
az acr create \
  --resource-group microservices-rg \
  --name sliitmicroregistry \
  --sku Basic
```

The registry name must be globally unique across all Azure customers. If sliitmicroregistry is taken, append your student ID — e.g. sliitmicroregistry20220456.

📝 **Note:** *The Basic SKU is sufficient for this lab. In production, Standard or Premium SKUs add features like geo-replication, content trust, and higher throughput.*

Step 2.3 – Authenticate Docker with ACR

Log your local Docker client into the registry so you can push images:

```
az acr login --name sliitmicroregistry
```

You should see: Login Succeeded

Step 2.4 – Verify in Azure Portal

Optionally, navigate to https://portal.azure.com and confirm:

1. Search for "Resource groups" in the top search bar
2. Click on microservices-rg
3. Verify sliitmicroregistry appears in the list of resources

Verification Checklist

- ☐ Resource group microservices-rg shows provisioningState: Succeeded
- ☐ Container registry sliitmicroregistry was created successfully
- ☐ az acr login returns Login Succeeded
- ☐ Registry is visible in Azure Portal under the resource group

Task 3 – Build & Push Docker Images

Objective

Build a Docker image for the gateway microservice and push it to your Azure Container Registry. This image will be used in Task 4 to run the service in the cloud.

Background

Docker images are portable, self-contained packages that include your application code and all its runtime dependencies. By tagging an image with your ACR login server address (e.g. sliitmicroregistry.azurecr.io), Docker knows where to push it when you run docker push.

Step-by-Step Instructions

Step 3.1 – Clone the Sample Repository

If you have not already done so, clone the sample application:

```
git clone https://github.com/your-org/azure-microservices-lab.git
cd azure-microservices-lab
```

📝 **Note:** *Replace the URL above with the actual repository URL provided by your instructor. The repo should contain a /gateway folder with a Dockerfile.*

Step 3.2 – Review the Dockerfile

Before building, review the gateway Dockerfile:

```
cat ./gateway/Dockerfile
```

A typical Node.js gateway Dockerfile looks like:

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Step 3.3 – Build the Docker Image

Build the image and tag it with your ACR login server address:

```
docker build -t sliitmicroregistry.azurecr.io/gateway:v1 ./gateway
```

Breaking down this command:

- docker build – builds an image from a Dockerfile
- -t sliitmicroregistry.azurecr.io/gateway:v1 – names and tags the image
- ./gateway – the build context (directory containing the Dockerfile)

💡 **Tip:** Add --no-cache to force a fresh build if you have made changes: docker build --no-cache -t sliitmicroregistry.azurecr.io/gateway:v1 ./gateway

Step 3.4 – Verify the Image Was Built

List your local Docker images to confirm the build succeeded:

```
docker images | grep gateway
```

You should see a row with your image name, the v1 tag, and a recent creation timestamp.

Step 3.5 – Push the Image to ACR

Push the image to your Azure Container Registry:

```
docker push sliitmicroregistry.azurecr.io/gateway:v1
```

Docker will upload each image layer. This may take 1–3 minutes depending on your internet connection. You will see progress output for each layer.

Step 3.6 – Verify the Image in ACR

Confirm the image was pushed successfully:

```
az acr repository list --name sliitmicroregistry --output table
```

Then view the tags for the gateway repository:

```
az acr repository show-tags --name sliitmicroregistry --repository gateway --output table
```

Verification Checklist

- ☐ docker build completed without errors
- ☐ docker images shows sliitmicroregistry.azurecr.io/gateway:v1
- ☐ docker push completed successfully for all layers
- ☐ az acr repository list shows gateway in the results
- ☐ Tag v1 is visible in the repository

Task 4 – Deploy Container App

Objective

Register the Azure Container Apps provider, create a managed environment, and deploy the gateway Docker image as a publicly accessible Azure Container App.

Background

Azure Container Apps is a serverless platform for running containerized applications. Unlike Azure Kubernetes Service (AKS), it abstracts away infrastructure management. You define your container, set scaling rules, and Azure handles the rest. Container Apps run within an Environment, which provides shared networking and observability.

Step-by-Step Instructions

Step 4.1 – Register the Container Apps Provider

Azure requires the Microsoft.App resource provider to be registered on your subscription before using Container Apps:

```
az provider register --namespace Microsoft.App --wait
```

The --wait flag holds the terminal until registration is complete (usually 1–2 minutes).

Note: *Also register the OperationalInsights provider, which is used for logging: az provider register --namespace Microsoft.OperationalInsights --wait*

Step 4.2 – Create the Container Apps Environment

An Environment is the shared runtime for your container apps. Create one now:

```
az containerapp env create \
  --name micro-env \
  --resource-group microservices-rg \
  --location eastus
```

This provisions the managed environment. It may take 2–3 minutes. Wait for the command to return before proceeding.

Step 4.3 – Enable ACR Admin Credentials

Container Apps needs credentials to pull your image from ACR. Enable admin access:

```
az acr update -n sliitmicroregistry --admin-enabled true

# Retrieve the credentials
```

```
az acr credential show --name sliitmicroregistry
```

Note down the username and one of the passwords from the output.

Step 4.4 – Deploy the Container App

Deploy the gateway container using the image you pushed in Task 3:

```
az containerapp create \
  --name gateway \
  --resource-group microservices-rg \
  --environment micro-env \
  --image sliitmicroregistry.azurecr.io/gateway:v1 \
  --target-port 3000 \
  --ingress external \
  --registry-server sliitmicroregistry.azurecr.io \
  --registry-username sliitmicroregistry \
  --registry-password <your-acr-password>
```

Key parameters explained:

- --target-port 3000 – the port your application listens on inside the container
- --ingress external – makes the app publicly accessible via HTTPS
- --registry-server / --registry-username / --registry-password – ACR auth

Step 4.5 – Retrieve the Application URL

Once deployed, get the public URL of your container app:

```
az containerapp show \
  --name gateway \
  --resource-group microservices-rg \
  --query properties.configuration.ingress.fqdn \
  --output tsv
```

Copy the returned URL (e.g. gateway.kindbeach-12345abc.eastus.azurecontainerapps.io) and open it in your browser. You should see your gateway service response.

Verification Checklist

- ☐ Microsoft.App provider registered successfully
- ☐ Container Apps Environment micro-env created
- ☐ Container app gateway deployed without errors
- ☐ Public FQDN URL is returned by az containerapp show
- ☐ Gateway service is accessible in the browser via HTTPS

⚠ **Warning:** If the app returns a 503 error, wait 2–3 minutes and retry. Container Apps may take time to pull the image and initialize the first replica.

Task 5 – Deploy Static Web App Frontend

Objective

Deploy the frontend application using Azure Static Web Apps, a fully managed hosting service optimized for static assets and single-page applications (SPAs).

Background

Azure Static Web Apps automatically builds and deploys full-stack web applications from a GitHub repository. It provides global CDN distribution, free SSL/TLS certificates, and seamless integration with Azure Functions for serverless APIs. For this lab, we use the CLI approach to create the resource, then link it to a GitHub repo.

Step-by-Step Instructions

Step 5.1 – Prepare Your Frontend

Ensure your frontend is in a GitHub repository. The repo should contain:

- An index.html (or a package.json for a React/Vue/Angular app)
- A build output folder (typically /build or /dist for compiled frameworks)

Tip: If you are using a simple HTML frontend, place your files in a /public folder at the root of the repo.

Step 5.2 – Create the Static Web App Resource

Run the following command. You will be prompted to authenticate with GitHub:

```
az staticwebapp create \
  --name sliit-frontend-app \
  --resource-group microservices-rg \
  --location eastus \
  --source https://github.com/<your-username>/<your-repo> \
  --branch main \
  --app-location "/" \
  --output-location "build"
```

Replace <your-username>/<your-repo> with your actual GitHub repository path. The --app-location is the root of your frontend source and --output-location is where the build output is generated.

Step 5.3 – Monitor the Deployment

The Static Web App creates a GitHub Actions workflow that automatically builds and deploys your app. Monitor the deployment:

4. Go to your GitHub repository in the browser

5. Click on Actions tab
6. Watch the Azure Static Web Apps CI/CD workflow run
7. Wait for the green checkmark (usually takes 2–5 minutes)

Step 5.4 – Get the Deployment URL

Retrieve the URL of your deployed frontend:

```
az staticwebapp show \
  --name sliit-frontend-app \
  --resource-group microservices-rg \
  --query defaultHostname \
  --output tsv
```

Open the returned URL in your browser to verify the frontend is live.

Step 5.5 – Connect Frontend to Gateway (Optional)

Update your frontend configuration to point API calls to the gateway URL from Task 4. In your frontend code, set the API base URL environment variable:

```
az staticwebapp appsettings set \
  --name sliit-frontend-app \
  --resource-group microservices-rg \
  --setting-names REACT_APP_API_URL=https://<gateway-fqdn>
```

Verification Checklist

- ☐ Static Web App resource created successfully in Azure
- ☐ GitHub Actions workflow ran and completed successfully
- ☐ Frontend is accessible via the assigned Azure URL
- ☐ Page renders without errors in the browser console

Task 6 – Verify Deployment & Cleanup

Objective

Perform end-to-end verification of your entire deployment to confirm all services are running as expected, then delete all Azure resources to avoid incurring ongoing costs.

Verification Steps

Step 6.1 – Verify All Resources

List all resources in your resource group:

```
az resource list --resource-group microservices-rg --output table
```

You should see at minimum:

- Container Registry – sliitmicroregistry (Microsoft.ContainerRegistry/registries)
- Container Apps Environment – micro-env (Microsoft.App/managedEnvironments)
- Container App – gateway (Microsoft.App/containerApps)
- Static Web App – sliit-frontend-app (Microsoft.Web/staticSites)

Step 6.2 – Test the Gateway Endpoint

Use curl or your browser to verify the gateway returns a valid response:

```
curl https://<your-gateway-fqdn>/health
```

Expected: HTTP 200 with a JSON response like { "status": "ok" }

Step 6.3 – Test the Frontend

Open the Static Web App URL in your browser and verify:

- The page loads without errors
- The browser console (F12 > Console) shows no critical errors
- Any API calls to the gateway succeed (check the Network tab)

Step 6.4 – Review Container App Logs

Inspect the container app logs to confirm it started correctly:

```
az containerapp logs show \
  --name gateway \
  --resource-group microservices-rg \
  --follow false
```

Cleanup – Delete All Resources

When you are finished with the lab, delete all provisioned resources to avoid unexpected charges on your Azure subscription:

```
az group delete --name microservices-rg --yes
```

The --yes flag skips the confirmation prompt. This operation is irreversible and will delete all resources in the group including the Container Registry, Container Apps, and Static Web App.

⚠ **Warning:** This command permanently deletes all resources in the microservices-rg resource group. Do NOT run this until you have completed the lab and taken any required screenshots for your submission.

Final Verification Checklist

- ☐ All 4 resources are visible in az resource list output
- ☐ Gateway HTTPS endpoint returns a valid response
- ☐ Frontend loads correctly in the browser
- ☐ No errors appear in the container app logs
- ☐ All required screenshots have been saved for submission
- ☐ az group delete was run to clean up all resources

Troubleshooting Guide

Use this guide to diagnose and resolve common issues encountered during the lab.

Problem	Possible Cause	Solution
az login not working	Not authenticated / session expired	Run az login again. In headless environments, use az login --use-device-code
docker push fails	ACR login missing or expired	Run az acr login --name sliitmicroregistry to refresh authentication
Container not accessible	Port not exposed or ingress not set	Verify --target-port matches your app's EXPOSE port and --ingress is set to external
Static web app deploy fails	GitHub Actions authentication issue	Re-authenticate: az staticwebapp secrets reset-api-key and re-link repo
Container app returns 503	Image pull failure or app startup error	Check logs with az containerapp logs show and verify ACR credentials
Provider registration error	Insufficient subscription permissions	Contact your subscription admin to register Microsoft.App namespace
Resource already exists	Duplicate name in same region	Append a unique suffix (e.g., student ID) to the resource name
ACR push: denied	Admin credentials not enabled	Run az acr update -n sliitmicroregistry --admin-enabled true

Useful Commands Reference

Keep these commands handy throughout the lab:

```
# Check login status
az account show

# List all resources in group
az resource list --resource-group microservices-rg --output table

# View container app details
az containerapp show --name gateway --resource-group microservices-rg

# Stream live logs
az containerapp logs show --name gateway --resource-group microservices-rg --follow true

# Restart container app
az containerapp revision restart --name gateway --resource-group microservices-rg \
  --revision <revision-name>

# List ACR images
az acr repository list --name sliitmicroregistry --output table
```